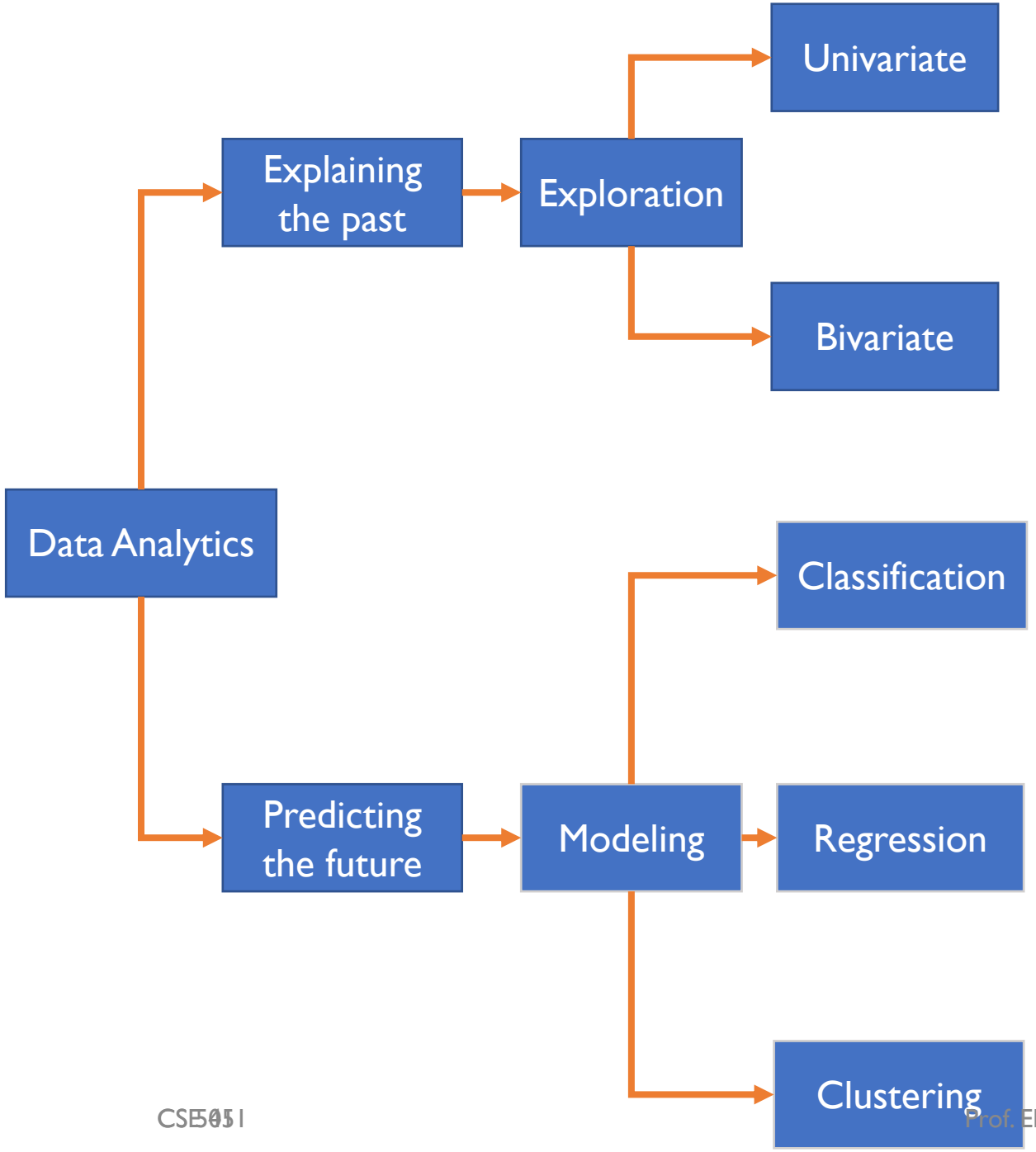
Beyond Numbers: Natural Language Processing

CSE 451
Topics in Computer Science
Fall 2023

© Royal Military College of Canada


Data Analytics
Explaining the past
Exploration
Univariate
Bivariate
Predicting the future
Modeling
Classification
Regression
Clustering

DONE

What is Natural Language Processing?

- Field of study focused on making sense of language
  - Using statistics and computers.
- You will learn the basics of NLP
  - Topic identification
  - Text classification
- NLP applications include:
  - Chatbots
  - Translation
  - Sentiment analysis
  - … and many more

Part 1

- Regular expressions
- Tokenization with NLTK and regex
- Charting word length with NLTK

What exactly are regular expressions?

- Strings with a special syntax
- Allow us to match patterns in other strings
- Applications of regular expressions:
  - Find all web links in a document
  - Parse email addresses
  - Remove/replace unwanted characters

- In Python

```
import re
re.match('abc', 'abcde')

word_regex = '\w+'
re.match(word_regex,
        'Hi there!')
```

<re.Match object; span=(0, 3), match='abc'>

<re.Match object; span=(0, 2), match='Hi'>

Common regex patterns

Pattern	Matches	Example
\w+	Word	‘Magic’
\d	Digit	9
\s	Space	‘ ’
.*	Wildcard	‘username74’
\S	Not space	‘no_spaces’
[a-z]	Lowercase group	‘abcdefg’

Python's re module

- `re` module
- `split`: split a string on regex
- `findall`: find all patterns in a string
- `search`: search for a pattern
- `match`: match an entire string or substring based on a pattern

- Pattern first, and the string second
- May return an iterator, string, or match object.

```
re.split('\s+', 'Split on spaces')
```

['Split', 'on', 'spaces']

What is tokenization?

- Turning a string or document into tokens (smaller chunks)
- One step in preparing a text for NLP
- Many different theories and rules
- You can create your own rules using regular expressions
- Some examples:
  - Breaking out words or sentences
  - Separating punctuation
  - Separating all hashtags in a tweet

NLTK Library

- nltk : natural language toolkit

```
from nltk.tokenize import word_tokenize
word_tokenize("Hi there!")
```

['Hi', 'there', '!']

Why tokenize?

- Easier to map part of speech
- Matching common words
- Removing unwanted tokens
- "I don't like Sam's shoes.”
- "I", "do", "n't", "like", "Sam", "'s", "shoes", "."

Other NLTK tokenizers

- `sent_tokenize`: tokenize a document into sentences
- `regexp_tokenize`: tokenize a string or document based on a regular expression pattern
- `TweetTokenizer`: special class just for tweet tokenization, allowing you to separate hashtags, mentions and lots of exclamation points!!!

More regex practice

- Difference between `re.search()` and `re.match()`

```
import re
re.match('abc', 'abcde')
<_sre.SRE_Match object; span=(0, 3), match='abc'>

re.search('abc', 'abcde')
<_sre.SRE_Match object; span=(0, 3), match='abc'>

re.match('cd', 'abcde’)
None

re.search('cd', 'abcde')
<_sre.SRE_Match object; span=(2, 4), match='cd'>
```

More Tokenization: Regex groups using or "|"

- OR is represented using |
- You can define a group using ()
- You can define explicit character ranges using []

```
import re
match_digits_and_words = ('(\d+|\w+)')
re.findall(match_digits_and_words, 'He has 11 cats.')
['He', 'has', '11', 'cats']
```

Regex ranges and groups

pattern	matches	example
[A-Za-z]+	upper and lowercase English alphabet	'ABCDEFGhijk'
[0-9]	numbers from 0 to 9	9
[A-Za-z-.]+	upper and lowercase English alphabet, - and .	'My-Website.com'
(a-z)	a, - and z	'a-z'
(\s+,)	spaces or a comma	' ,'

Character range with `re.match()`

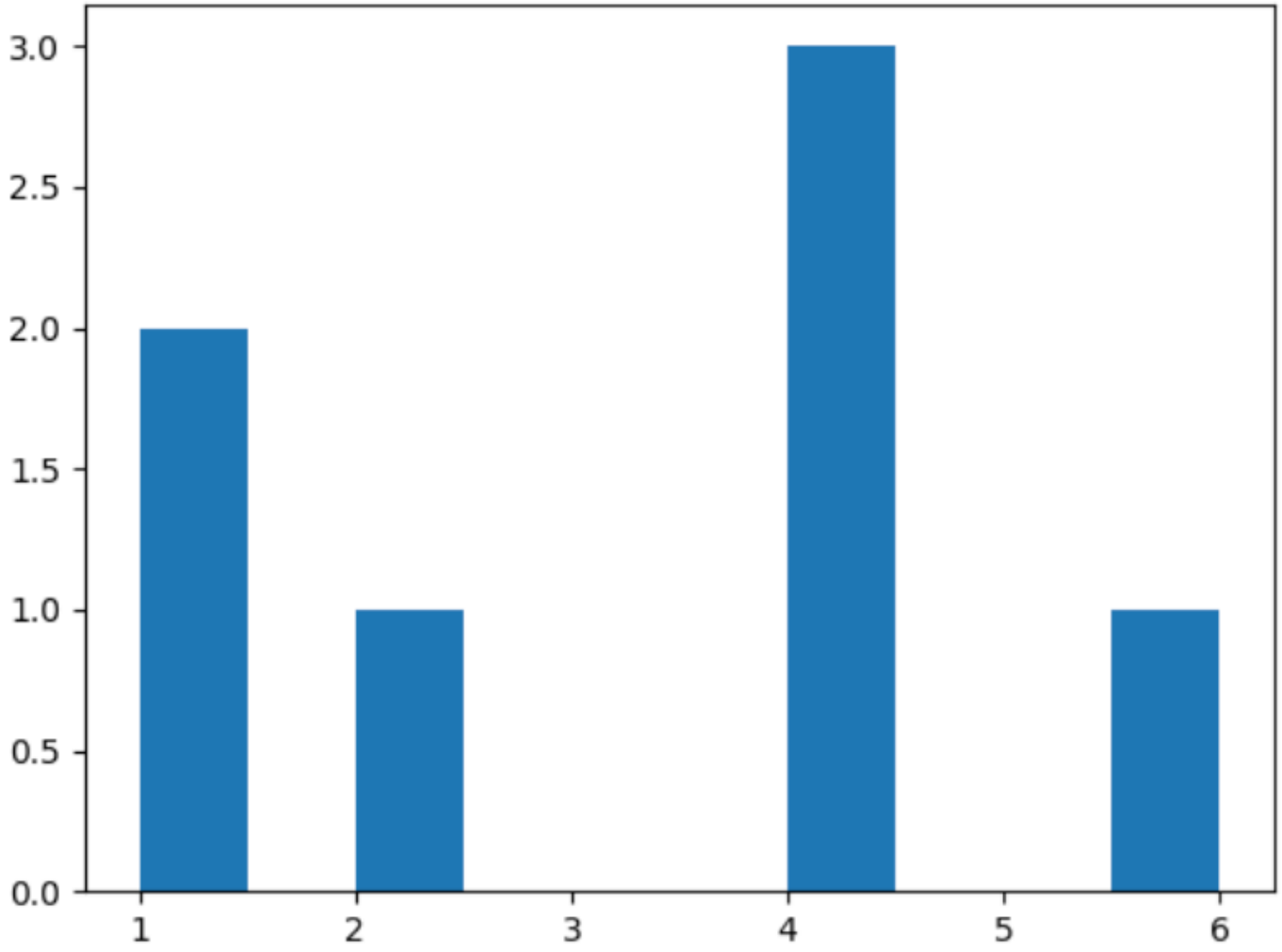
```
import re
my_str = 'match lowercase spaces nums like 12, but no commas'
re.match('[a-z0-9 ]+', my_str)
```

<re.Match object; span=(0, 42), match='match lowercase spaces nums like 12'>

Charting word length with nltk and matplotlib

```
from matplotlib import pyplot as plt
from nltk.tokenize import word_tokenize
words = word_tokenize("This is a pretty cool tool!")
word_lengths = [len(w) for w in words]
plt.hist(word_lengths)
plt.show()
```

Word length histogram



Part 2: Simple Topic Identification

- Word counts with bag-of-words
- Simple text processing
- Introduction to gensim
- Tf-idf with gensim

Bag-of-words

- Basic method for finding topics in a text
- Need to first create tokens using tokenization
- ...and then count up all the tokens
- The more frequent a word, the more important it might be
- Can be a great way to determine the significant words in a text

- Can be used for various tasks such as document classification, topic modeling, and sentiment analysis.

Bag-of-words example

- Text: "The cat is in the box. The cat likes the box. The box is over the cat."
- Bag of words (stripped punctuation):
  - "The": 3, "box": 3
  - "cat": 3, "the": 3
  - "is": 2
  - "in": 1, "likes": 1, "over": 1

Bag-of-words in Python

```
from nltk.tokenize import word_tokenize
from collections import Counter
Counter(word_tokenize("""The cat is in the box. The cat Likes the
             box. The box is over the cat."""))
```

Counter({'.' : 3, 'The': 3, 'box': 3, 'cat': 3, 'in': 1, ... 'the': 3})

```
counter.most_common(2)
```

[('The', 3), ('box', 3)]

Simple text pre-processing: why pre-process?

- Helps make for better input data
  - When performing machine learning or other statistical methods
- Examples:
  - Tokenization to create a bag of words
  - Lowercasing words
- Lemmatization/Stemming
  - Shorten words to their root stems
- Removing stop words, punctuation, or unwanted tokens
- Good to experiment with different approaches

Text preprocessing with Python

```
from nltk.corpus import stopwords
text = """The cat is in the box. The cat likes the box. The box is
over the cat."""

tokens = [w for w in word_tokenize(text.lower())
                        if w.isalpha()]
no_stops = [t for t in tokens if t not in
                        stopwords.words('english')]
Counter(no_stops).most_common(2)
```

[('cat', 3), ('box', 3)]

Introduction to gensim

- Popular open-source NLP library
- Uses top academic models to perform complex tasks
  - Building document or word vectors

What is a word vector?

- A word embedding or vector is trained from a large corpus
- A word vector is a multi-dimensional representation of a word or document.

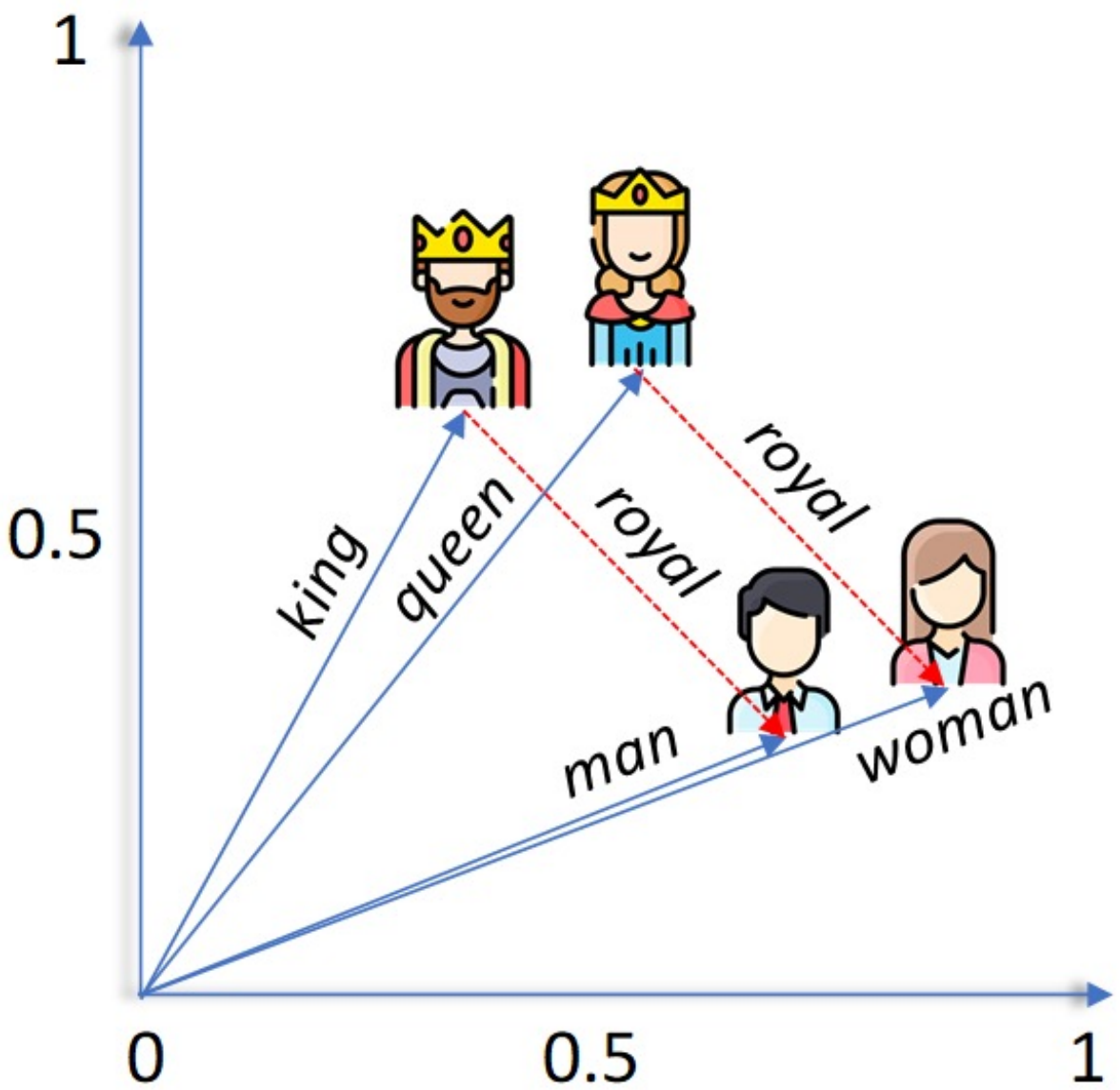
- Think of it as a multi-dimensional array normally with sparse features (lots of zeros and some ones).

- With these vectors, we can then see relationships among the words or documents based on how near or far they are.

Word vector

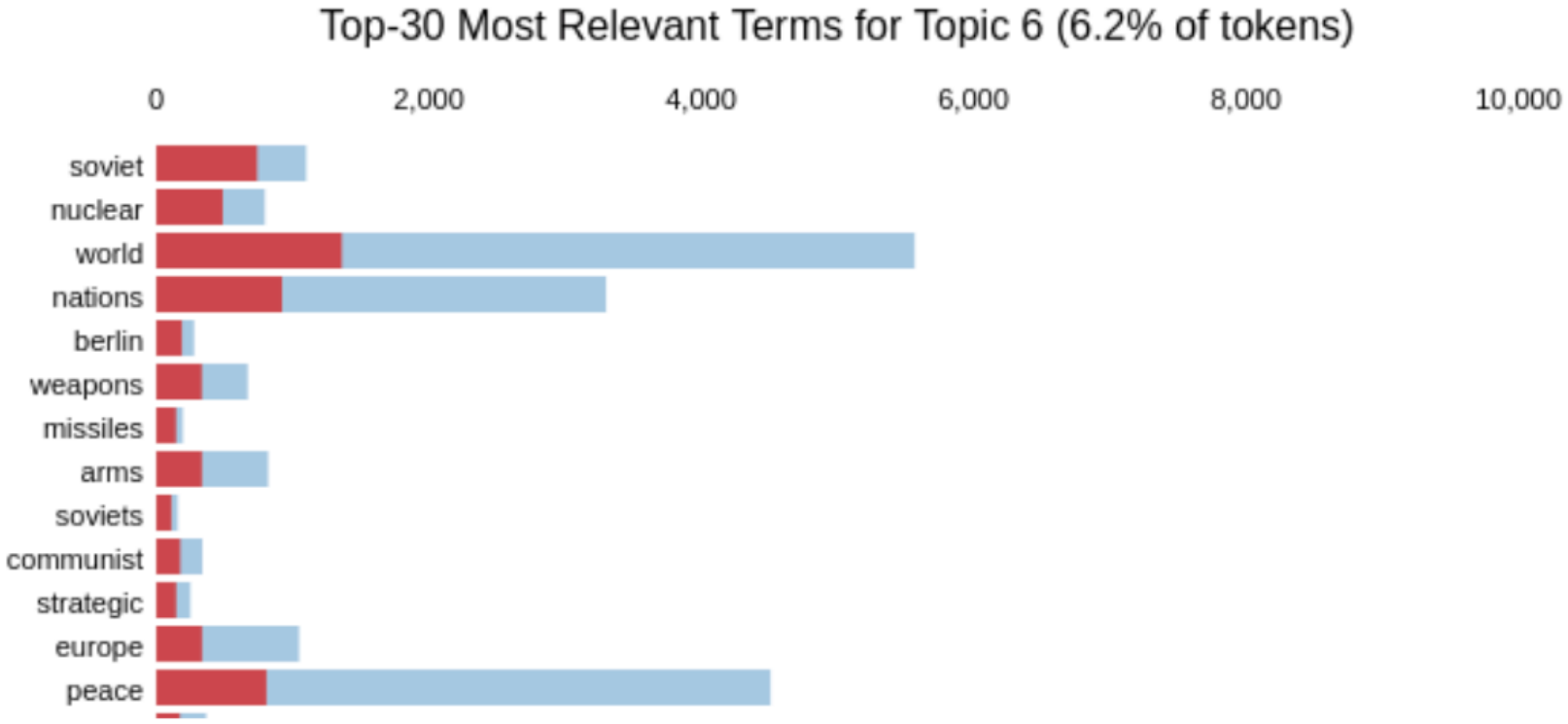
king - man + woman = queen

The deep learning algorithm used to create word vectors has been able to distill this meaning based on how those words are used throughout the text.



gensim example

The graphic we have here is an example of LDA visualization. LDA stands for latent dirichlet allocation, and it is a statistical model we can apply to text using Gensim for topic analysis and modelling.



Source: http://tlfvincent.github.io/2015/10/23/presidential-speech-topics

Creating a gensim dictionary

- Gensim allows you to build corpora and dictionaries using simple classes and functions.
- A corpus (or if plural, corpora) is a set of texts used to help perform natural language processing tasks.

```python
from gensim.corpora.dictionary import Dictionary
from nltk.tokenize import word_tokenize

# Sample documents
my_documents = [
    'The movie was about a spaceship and aliens.',
    'I really liked the movie!',
    'Awesome action scenes, but boring characters.',
    'The movie was awful! I hate alien films.',
    'Space is cool! I liked the movie.',
    'More space films, please!'
]

# Tokenize the documents into words
tokenized_docs = [word_tokenize(doc.lower()) for doc in my_documents]

# Create a dictionary from the tokenized documents
dictionary = Dictionary(tokenized_docs)

# Display the token-to-ID mapping
token_to_id = dictionary.token2id
for token, token_id in token_to_id.items():
    print(f'{token}: {token_id}')
```

Creating a gensim corpus

- Using the dictionary we built in the last slide, we can then create a Gensim corpus. This is a bit different than a normal corpus -- which is just a collection of documents.

- Gensim uses a simple bag-of-words model which transforms each document into a bag of words using the token ids and the frequency of each token in the document.

Creating a gensim corpus

```
corpus = [dictionary.doc2bow(doc) for doc in tokenized_docs]
corpus
```

```
[[(0, 1), (1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1)],
 [(0, 1), (1, 1), (9, 1), (10, 1), (11, 1), (12, 1)],
 ...
]
```

- gensim models can be easily saved, updated, and reused
- dictionary can also be updated
- This more advanced and feature rich bag-of-words can be used in future exercises

TF-IDF with genism: What is TF-IDF?

- Term frequency - inverse document frequency
- Allows you to determine the most important words in each document
- Each corpus may have shared words beyond just stopwords
- These words should be down-weighted in importance
- Example from astronomy: "Sky”
- Ensures most common words don't show up as key words
- Keeps document specific frequent words weighted high

TF-IDF Formula

$$w_{i,j} = tf_{i,j} * \log(\frac{N}{df_i})$$

$$w_{i,j} = \text{tf-idf weight for token } i \text{ in document } j$$

$$tf_{i,j} = \text{number of occurences of token } i \text{ in document } j$$

$$df_i = \text{number of documents that contain token } i$$

$$N = \text{total number of documents}$$

TF-IDF with gensim

```
from gensim.models.tfidfmodel import TfidfModel

tfidf = TfidfModel(corpus)
tfidf[corpus[1]]
```

[(0, 0.1746298276735174),
(1, 0.1746298276735174),
(9, 0.2985316622146367 3),
(10, 0.7716931521027908), ...]

Part 3: Named Entity Recognition

- NER
- SpaCy

What is NER?

NLP task to identify important named entities in the text

- People, places, organizations
- Dates, states, works of art
- ... and other categories!

Can be used alongside topic identification

- ... or on its own!

- Who? What? When? Where?

Example of NER

In 1917, Einstein applied the general theory of relativity to model the large-scale structure of the universe. He was visiting the United States when Adolf Hitler came to power in 1933 and did not go back to Germany, where he had been a professor at the Berlin Academy of Sciences. He settled in the U.S., becoming an American citizen in 1940. On the eve of World War II, he endorsed a letter to President Franklin D. Roosevelt alerting him to the potential development of "extremely powerful bombs of a new type" and recommending that the U.S. begin similar research. This eventually led to what would become the Manhattan Project. Einstein supported defending the Allied forces, but largely denounced using the new discovery of nuclear fission as a weapon. Later, with the British philosopher Bertrand Russell, Einstein signed the Russell-Einstein Manifesto, which highlighted the danger of nuclear weapons. Einstein was affiliated with the Institute for Advanced Study in Princeton, New Jersey, until his death in 1955.

Tag colours:
LOCATION TIME PERSON ORGANIZATION MONEY PERCENT DATE

- The text has been highlighted for different types of named entities that were found using the Stanford NER library.
- You can use NER to solve problems like fact extraction as well as which entities are related using computational language models.
  - For example, in this text we can see that Einstein has something to do with the United States, Adolf Hitler and Germany.
  - We can also see by token proximity that Betrand Russel and Einstein created the Russel-Einstein manifesto -- all from simple entity highlighting.

nltk and the Stanford CoreNLP Library

- NLTK allows you to interact with named entity recognition via its own model, but also the aforementioned Stanford library.

- The Stanford CoreNLP library integrates into Python via nltk.
- Integration requires you to perform a few steps before you can use it, including installing the required Java files and setting system environment variables.
- Can also be used on its own.

- Has great support for NER, as well as some related nlp tasks such as coreference (linking pronouns and entities together) and dependency trees.

Using nltk for Named Entity Recognition

```
import nltk
sentence = "'In New York, I like to ride the Metro to visit MOMA and
             some restaurants rated well by Ruth Reichl.'"
tokenized_sent = nltk.word_tokenize(sentence)
tagged_sent = nltk.pos_tag(tokenized_sent) # This will add tags for
proper nouns, pronouns, adjective, verbs and other part of speech
that NLTK uses based on an english grammar.
tagged_sent[:3]
```

```
[('In', 'IN'), ('New', 'NNP'), ('York', 'NNP')]
```

Then we pass this tagged sentence into the ne_chunk function, or named entity chunk, which will return the sentence as a tree.

```
print(nltk.ne_chunk(tagged_sent))
```

Named Entity Types:

- GPE: Geopolitical entity.
- ORGANIZATION: This refers to named corporate, governmental, or other organizational entities.
- PERSON: This refers to names of people.

POS tags

- NNP: Proper noun, singular
- IN: Preposition or subordinating conjunction
- VBP: Verb, non-3rd person singular present
- TO: "to" as preposition or infinitive marker
- VB: Verb, base form
- DT: Determiner

```
(S
  In/IN
  (GPE New/NNP York/NNP)
  ,/,
  I/PRP
  like/VBP
  to/TO
  ride/VB
  the/DT
  (ORGANIZATION Metro/NNP)
  to/TO
  visit/VB
  (ORGANIZATION MOMA/NNP)
  and/CC
  some/DT
  restaurants/NNS
  rated/VBN
  well/RB
  by/IN
  (PERSON Ruth/NNP Reichl/NNP)
  ./.)
```

Introduction to SpaCy

- SpaCy is a NLP library similar to Gensim, but with different implementations.
- Includes a particular focus on creating NLP pipelines to generate models and corpora.

- SpaCy is open-source and has several extra libraries and tools built by the same team,
  - Displacy - a visualization tool for viewing parse trees which uses Node-js to create interactive text.

DisplaCy entity recognition visualizer

- https://demos.explosion.ai/displacy-ent

```python
import spacy
nlp = spacy.load('en_core_web_sm')
nlp.entity
```

```
<spacy.pipeline.EntityRecognizer at 0x7f76b75e68b8>
```

```python
doc = nlp("""Berlin is the capital of Germany;
                  and the residence of Chancellor Angela Merkel.""")
doc.ents
```

```
(Berlin, Germany, Angela Merkel)
```

```python
print(doc.ents[0], doc.ents[0].label_)
```

```
Berlin GPE
```

Why use SpaCy for NER?

- Ease pipeline creation
- Different entity types compared to nltk
- Informal language corpora
  - Easily finds entities in tweets and chat messages
- Quickly growing.

For multilingual support of NER

- The polyglot library

Part 4: Building a “Fake news classifier”

- apply the NLP basics with some supervised machine learning to build a "fake news" detector.

- Dataset: https://www.kaggle.com/datasets/hassanamin/textdb3